

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN ENXEÑARÍA DE COMPUTADORES



Sistema de captura y procesado de datos IMU en tiempo real para estimación de movimiento vertical

Estudiante: Mateo Rivela Santos

Dirección: Dirección por asignar

A Coruña, abril de 2026.

A mi familia, por su apoyo constante en cada etapa de este camino, y a todas las personas que me animaron a seguir aprendiendo cuando el resultado no salía a la primera.

Agradecimientos

Agradezco a mi entorno personal por la paciencia y el apoyo durante el desarrollo de este trabajo. Agradezco también al profesorado de la FIC por la base técnica y metodológica que ha hecho posible este proyecto, y a las personas que me ayudaron con pruebas, validaciones y revisión de resultados.

Resumen

Este trabajo presenta el desarrollo de un producto software móvil para captura y procesamiento de datos inerciales, con Android como plataforma ubicua principal. El sistema se diseña con dos modos de operación: modo autónomo (captura y procesamiento en el propio teléfono) y modo pantalla (visualización de resultados procesados por una plataforma externa como ESP32+MPU6050).

La solución implementada integra captura de sensores, normalización temporal, cálculo de métricas de oleaje y visualización en una misma aplicación, además de exportación de sesiones con parámetros y resultados. El algoritmo de procesamiento se utiliza como componente técnico del producto, no como fin en sí mismo.

Los resultados obtenidos muestran que la arquitectura de plataforma es viable y extensible, y permiten definir una comparativa homogénea entre Android y ESP32 para evaluar rendimiento, robustez y limitaciones en escenarios equivalentes.

Abstract

Este traballo presenta o desenvolvemento dun produto software móbil para captura e procesado de datos inerciais, con Android como plataforma ubicua principal. O sistema deseñouse con dous modos de operación: modo autónomo (captura e procesado no propio teléfono) e modo pantalla (visualización de resultados procesados por unha plataforma externa como ESP32+MPU6050).

A solución integra captura de sensores, normalización temporal, cálculo de métricas de ondata, visualización e exportación de sesións con parámetros e resultados nunha mesma aplicación. O algoritmo de procesado emprégase como compoñente técnico do produto e non como obxectivo final por si mesmo.

Os resultados mostran que a arquitectura de plataforma é viable e extensible, e permiten definir unha comparativa homoxénea entre Android e ESP32 para avaliar rendemento, robustez e limitacións en escenarios equivalentes.

Palabras clave:

- Unidad de medida inercial (IMU)
- Android
- Plataforma ubicua
- Producto software
- Procesado on-device
- Arquitectura móvil
- Comparativa Android-ESP32

Keywords:

- Unidade de medida inercial (IMU)
- Android
- Plataforma ubicua
- Produto software
- Procesado on-device
- Arquitectura móbil
- Comparativa Android-ESP32

Índice general

1	Introducción	1
1.1	Motivación del problema	1
1.2	Objetivos	2
1.2.1	Objetivo general	2
1.2.2	Objetivos específicos	2
1.3	Alcance y limitaciones	3
1.4	Metodología de desarrollo	4
1.5	Estructura de la memoria	4
2	Estado del arte y contexto tecnológico	5
2.1	Caracterización del oleaje mediante parámetros estadísticos	6
2.2	Android como plataforma de sensado de bajo coste	7
2.2.1	Límites de una solución móvil pura	8
2.3	Nodos embebidos para adquisición dedicada	9
2.4	Marco comercial y soluciones profesionales de referencia	10
2.5	Posicionamiento del trabajo	11
3	Análisis del problema, requisitos, planificación y presupuesto	13
3.1	Análisis del problema	13
3.2	Requisitos del sistema	14
3.2.1	Requisitos funcionales	14
3.2.2	Requisitos no funcionales	14
3.2.3	Restricciones técnicas	15
3.3	Casos de uso principales	15
3.4	Metodología y planificación	16
3.4.1	Enfoque de trabajo	16
3.4.2	Fases del proyecto	16
3.5	Riesgos y estrategia de validación	17

3.6	Material y medios necesarios	17
3.7	Presupuesto orientativo	18
3.7.1	Recursos utilizados por el proyecto	18
3.7.2	Adquisiciones imputables al proyecto	18
4	Diseño de la plataforma	19
4.1	Entorno y herramientas de desarrollo	19
4.2	Vista general del sistema	20
4.3	Diseño de adquisición	21
4.3.1	Sesión como unidad de captura	21
4.3.2	Adquisición en Android	22
4.3.3	Adquisición en ESP32	22
4.3.4	Separación entre captura y procesado	23
4.4	Diseño de comunicación	24
4.4.1	Contrato de control	24
4.4.2	Enlace entre Android y ESP32	24
4.4.3	Flujo de datos del sistema	25
4.5	Diseño de persistencia y backend	25
4.6	Diseño de interfaz y control	26
5	Implementación del sistema	30
5.1	Criterio de implementación incremental	30
5.2	Estado de partida de la implementación	31
5.3	Herramientas y organización del código	31
5.4	Fase 1: scripts de captura y análisis	32
5.5	Fase 2: implementación del modo móvil en Android	32
5.5.1	Estructura general de la aplicación	33
5.5.2	Captura de sensores	33
5.5.3	Persistencia local de sesiones	34
5.5.4	Gestión local y operaciones de depuración	34
5.5.5	Prototipo de interfaz auxiliar	35
5.6	Fase 3: procesado on-device	36
5.7	Fase 4: pruebas iniciales con ESP32 e IMU	37
5.8	Fase 5: firmware en C/C++ para ESP32	37
5.9	Fase 6: formato común de sesión y resultados	38
5.10	Fase 7: backend remoto	39
5.11	Criterios de cierre de implementación	40

6 Validación y evaluación	41
6.1 Metodología de validación	41
6.2 Escenarios de prueba	41
6.3 Configuración de pruebas	42
6.4 Métricas de evaluación	42
6.5 Plantilla de comparativa móvil vs embebido	43
6.6 Criterios de aceptación	43
6.7 Resultados	43
6.7.1 Resultados del modo móvil	43
6.7.2 Resultados del modo embebido	43
6.7.3 Evaluación del subsistema de comunicación	44
6.7.4 Resultados de integración por contratos	44
6.7.5 Comparación de operación entre modos	45
6.8 Discusión	45
6.8.1 Limitaciones del trabajo	46
6.8.2 Lecciones aprendidas	46
7 Conclusiones	47
7.1 Conclusiones principales	47
7.2 Conclusión metodológica	47
7.3 Implicaciones prácticas	48
7.4 Aportaciones	48
7.5 Trabajo futuro	48
7.5.1 Línea prioritaria inmediata	48
A Material adicional	50
A.1 Scripts principales	50
A.2 Comandos de referencia	50
A.2.1 Captura	50
A.2.2 Procesado rápido sin gráfica	50
A.2.3 Procesado paso a paso con validación temporal	50
A.3 Datos de prueba	51
Bibliografía	53

Índice de figuras

1.1	Esquema conceptual del problema abordado por el sistema.	2
2.1	Representación visual de algunos parámetros característicos del oleaje, como H_s , T_p y T_z . Fuente: Folkestone Fringe.	7
2.2	Referencia visual del smartphone como dispositivo de sensado, con integración de sensores y procesamiento local. Fuente: Medium.	8
2.3	Referencia visual del subsistema embebido de captura, formado por una placa de control/comunicación y una IMU externa. Fuente: Pinterest.	10
2.4	Ejemplos de soluciones profesionales de referencia en medida de oleaje y sensado inercial: familia de boyas Datawell Waverider (izquierda) y sensor VectorNav VN-200 (derecha). Fuentes: Datawell y VectorNav.	11
3.1	Casos de uso principales del sistema.	15
4.1	Arquitectura funcional del sistema con Android como plataforma central de operación.	21
4.2	Hueco para diagrama de arquitectura general del sistema.	21
4.3	Hueco para diagrama de adquisición en modo móvil y modo embebido.	23
4.4	Esquema del subsistema embebido y conexiones principales.	23
4.5	Hueco para diagrama de secuencia de operación remota.	24
4.6	Flujo de datos extremo a extremo del sistema.	25
4.7	Hueco para diagrama de despliegue lógico y persistencia.	26
4.8	Flujo Android: captura/operación, gestión de sesiones y visualización.	28
4.9	Flujo de comparativa entre nodo móvil y nodo embebido con contratos comunes.	29
4.10	Hueco para diagrama de estados de operación del sistema.	29
5.1	Pantalla real de la aplicación Android utilizada durante la depuración del modo móvil.	35

5.2	Prototipo de interfaz auxiliar usado para organizar las primeras pruebas de captura, gestión de sesiones y diagnóstico de modos de operación.	36
6.1	Escenarios de prueba del sistema.	42
6.2	Resultados representativos del modo móvil.	44
6.3	Resultados representativos del modo embebido.	44
6.4	Evaluación comparativa del subsistema de comunicación Android-ESP32. . . .	45
6.5	Validación de contratos de integración entre subsistemas.	45
6.6	Comparación operativa entre modo móvil y modo embebido.	45

Índice de cuadros

2.1	Comparación general entre smartphone, nodo embebido propio y soluciones profesionales de referencia.	12
2.2	Posicionamiento resumido del trabajo frente a enfoques habituales.	12
3.1	Primera estimación de los recursos utilizados por el proyecto, incluyendo medios ya disponibles.	18
3.2	Primera estimación del material comprado o imputable directamente al desarrollo del proyecto.	18
6.1	Plantilla de comparación entre modo móvil y modo embebido.	43

Introducción

ESTE trabajo fin de grado se centra en usar el teléfono móvil como plataforma ubicua para capturar y procesar datos inerciales, y en demostrar su viabilidad en un caso de uso de caracterización de oleaje. El interés del sistema no reside en un sensor concreto, sino en la posibilidad de articular una solución operativa y desplegable sobre plataformas accesibles.

1.1 Motivación del problema

En la actualidad, existen multitud de problemas relacionados con la monitorización de movimientos en general, ya sea aceleración, orientación o posición, que pueden ser abordados con sensores inerciales.

El fácil acceso a datos inerciales en condiciones reales, sin necesidad de equipamiento especializado, abre la puerta a aplicaciones de monitorización de estado del mar, navegación, salud o actividad física. Lo que se pretende con este TFG es explorar esta posibilidad usando algo tan común como un teléfono móvil y demostrar que, con un tratamiento de señal adecuado, es posible obtener métricas útiles a partir de datos inerciales brutos.

Gracias a la ubicuidad de los smartphones, la capacidad de transmitir información desde ubicaciones relativamente remotas y la potencia de procesamiento que ofrecen, se pueden desarrollar multitud de aplicaciones que aprovechen esos datos para caracterizar el entorno o el estado de un sistema físico. En este caso, se ha elegido el oleaje como caso de uso representativo, ya que es un fenómeno complejo que puede ser monitorizado con sensores inerciales y tiene aplicaciones prácticas en navegación, meteorología y ocio.

En este contexto, los sensores inerciales permiten explorar una alternativa ligera frente a instrumentación oceanográfica más compleja o costosa, siempre que el tratamiento de señal y la arquitectura global del sistema sean adecuados. También hay que considerar que dependiendo de la aplicación concreta, la calidad que permiten los smartphones pueden o no pueden ser suficientes, por lo que es importante validar esta aproximación y entender sus

limitaciones.



Figura 1.1: Esquema conceptual del problema abordado por el sistema.

[AÑADIDO] En este trabajo se adopta como método principal un enfoque tipo OpenMet-Buoy: estimación espectral mediante Welch, conversión de espectro de aceleración a elevación y cálculo de métricas de oleaje (H_s , T_z , T_c). La implementación se orienta a ejecución *on-device* en Android, con una fase posterior de comparación frente a ESP32+MPU6050.

1.2 Objetivos

1.2.1 Objetivo general

Diseñar e implementar una arquitectura modular de captura y gestión de datos inerciales, flexible y desacoplada del procesamiento aplicado, con Android como plataforma principal de operación y con capacidad de integración con un nodo embebido basado en ESP32+IMU.

1.2.2 Objetivos específicos

- Estudiar alternativas para la adquisición de datos en distintas plataformas, incluyendo dispositivos móviles y sistemas embebidos, analizando sus limitaciones y características en términos de precisión, estabilidad y coste.
- Implementar en Android un modo de captura, guardado y gestión de sesiones como base operativa del sistema.
- Implementar la integración con un nodo embebido basado en ESP32+IMU para consulta de estado, control de captura y descarga de sesiones.
- Definir contratos comunes de sesión, control y explotación para unificar ambos modos de operación y desacoplar la arquitectura del procesamiento aplicado.
- Evaluar la capacidad del sistema para integrarse con mecanismos de comunicación externa, supervisión y acceso remoto a datos, sin depender de tecnologías concretas.

- Aplicar la arquitectura desarrollada al caso práctico de caracterización del estado del mar, con el fin de validar su funcionamiento y analizar su comportamiento en condiciones reales o representativas de operación.

1.3 Alcance y limitaciones

El alcance de la memoria es el diseño e implementación de un sistema de adquisición y gestión de datos inerciales, no el cierre de una solución algorítmica definitiva para un único dominio de aplicación.

Dentro de este alcance se cubren la operación local en Android, la integración con un nodo embebido ESP32+IMU y la definición de interfaces comunes para evolucionar la arquitectura de forma controlada.

Las limitaciones principales consideradas son:

- **Variabilidad de sensores y temporización entre dispositivos Android.** Hay que tener en cuenta que diferentes modelos de teléfono presentan diferencias de hardware, calibración interna y frecuencia efectiva de muestreo. Esto claramente puede introducir variaciones en los datos capturados que si comprobamos la respuesta de ambos dispositivos incluso bajo condiciones similares van a mostrar resultados distintos. Esto debemos tenerlo en cuenta en la comparación entre sesiones, que requiere normalización y control de metadatos.
- **Estado iterativo de la integración hardware en el subsistema embebido.** El hardware siempre va a presetar una serie de problemas añadidos. En este caso la integración ESP32+IMU depende de aspectos prácticos de cableado, alimentación, estabilidad del bus , además de la introducción de estas en un entorno de prueba realista y tener en cuenta ese entorno para la propia construcción del sistema ,aunque para testear el sistema se ha optado por un entorno controlado, es importante tener en cuenta que el entorno real de uso puede introducir variaciones adicionales que deben ser consideradas en la evolución futura del sistema. Como consecuencia, parte del trabajo embebido se desarrolla de forma iterativa, combinando implementación y diagnóstico técnico.

[AÑADIDO]

- **Integración de backend cloud aún en fase de consolidación.** Aunque la arquitectura contempla sincronización y acceso remoto, la capa cloud todavía no está cerrada como componente definitivo del sistema. En esta fase, el foco se mantiene en estabilizar contratos de datos e interoperabilidad antes de fijar una solución de despliegue final.

- **Dependencia de campañas de prueba para cerrar validación comparativa completa.** La comparación entre plataformas requiere campañas repetibles con protocolo homogéneo de captura y operación. Mientras dichas campañas no estén completamente cerradas, las conclusiones comparativas deben considerarse provisionales y sujetas a validación adicional.
- **Restricciones de tiempo y alcance propias de un TFG.** El proyecto prioriza la definición arquitectónica, la implementación funcional y la trazabilidad técnica frente a una industrialización completa. Esto implica posponer algunas mejoras de robustez avanzada, automatización extensa y validación a gran escala para trabajo futuro.

1.4 Metodología de desarrollo

El trabajo sigue un enfoque incremental y modular, orientado a reducir riesgo técnico y a validar el sistema por fases. La descripción detallada de la metodología de desarrollo, planificación y validación se presenta en el Capítulo 3.

1.5 Estructura de la memoria

La memoria se organiza de la siguiente forma:

- En el Capítulo 2 se resume el estado del arte estrictamente necesario para justificar decisiones de producto.
- En el Capítulo 3 se describe la metodología de desarrollo, la planificación y el presupuesto de trabajo.
- En el Capítulo 4 se define la arquitectura funcional por capas y los contratos de interfaz.
- En el Capítulo 5 se documenta el estado real de implementación en modo móvil y embebido.
- En el Capítulo 6 se presenta la validación funcional, de integración y operativa del sistema.
- En el Capítulo 7 se recogen conclusiones, aportaciones y trabajo futuro.
- En el Apéndice A se incluye material complementario.

Estado del arte y contexto tecnológico

ESTE capítulo pretende situar el trabajo de fin de grado en el contexto de las tecnologías y enfoques existentes en el ámbito de los sistemas IoT con soporte móvil.

Para hablar de IoT es preciso hacer una revisión de las arquitecturas de aplicaciones IoT, centrándose en dónde se capturan, procesan, almacenan y gestionan los datos dentro de un sistema. En este sentido, existe una idea principal: no existe una arquitectura única que sea válida para todos los casos de uso. Cada contexto requiere una solución capaz de equilibrar necesidades distintas, como la latencia, la escalabilidad, el consumo o la privacidad.

Existen tres grandes enfoques:

- Cloud, que se centra en la captura de datos en el borde y su procesamiento en la nube, lo que permite una gran capacidad de procesamiento y almacenamiento, pero puede introducir latencia y dependencia de la conectividad.
- Edge, que se centra en el procesamiento de datos en el borde, cerca de la fuente de datos, lo que reduce la latencia y la dependencia de la conectividad, pero puede limitar la capacidad de procesamiento y almacenamiento.
- Fog, que se centra en el procesamiento de datos en la nube, pero con una capa intermedia de procesamiento en el borde, lo que permite un equilibrio entre la latencia y la capacidad de procesamiento.

Cada una de estas arquitecturas tiene ventajas e inconvenientes que, a la hora de diseñar un sistema IoT, deben tenerse en cuenta según las necesidades concretas de la aplicación.

2.1 Caracterización del oleaje mediante parámetros estadísticos

Una vez introducido el contexto arquitectónico general, resulta necesario describir el dominio de aplicación concreto considerado en este trabajo: la caracterización del oleaje.

Los parámetros estadísticos sirven para describir el estado del mar de una forma general pero útil. El oleaje es un fenómeno complejo y no resulta sencillo caracterizarlo con una sola medida instantánea. De acuerdo con la documentación técnica del National Data Buoy Center (NDBC), una forma habitual de describirlo consiste en emplear parámetros derivados del espectro de energía de la señal. Para este trabajo, los más relevantes son los siguientes:

- **Altura significativa (H_s):** se aproxima como la altura media del tercio más alto de las olas. En formulación espectral puede expresarse como:

$$H_s = 4\sqrt{m_0}$$

donde m_0 es el momento espectral de orden cero, que se obtiene integrando la densidad espectral de energía del oleaje:

$$m_0 = \int_0^\infty S(f)df$$

- **Periodo medio de cruce por cero (T_z):** en análisis espectral, esta magnitud suele aproximarse mediante los momentos espectrales como:

$$T_z = \sqrt{\frac{m_0}{m_2}}$$

donde m_2 es el momento espectral de orden dos. Este parámetro resume una escala temporal media del oleaje y resulta útil para comparar estados de mar sin necesidad de reconstruir cada ola individual.

- **Periodo pico (T_p):** se define como el periodo asociado a la frecuencia de máxima energía del espectro, y se calcula como:

$$T_p = \frac{1}{f_p}$$

donde f_p es la frecuencia con mayor densidad espectral.

El uso de estos parámetros refuerza una idea importante para este TFG: el objetivo no es reconstruir con precisión la forma instantánea de cada ola, sino obtener una caracterización

estadística y espectral del estado del mar a partir de señales inerciales. Por ello, el análisis del problema se apoya en magnitudes agregadas como H_s , T_z y T_p , que son habituales en la instrumentación oceanográfica y encajan de forma natural con un pipeline de procesado basado en espectros.

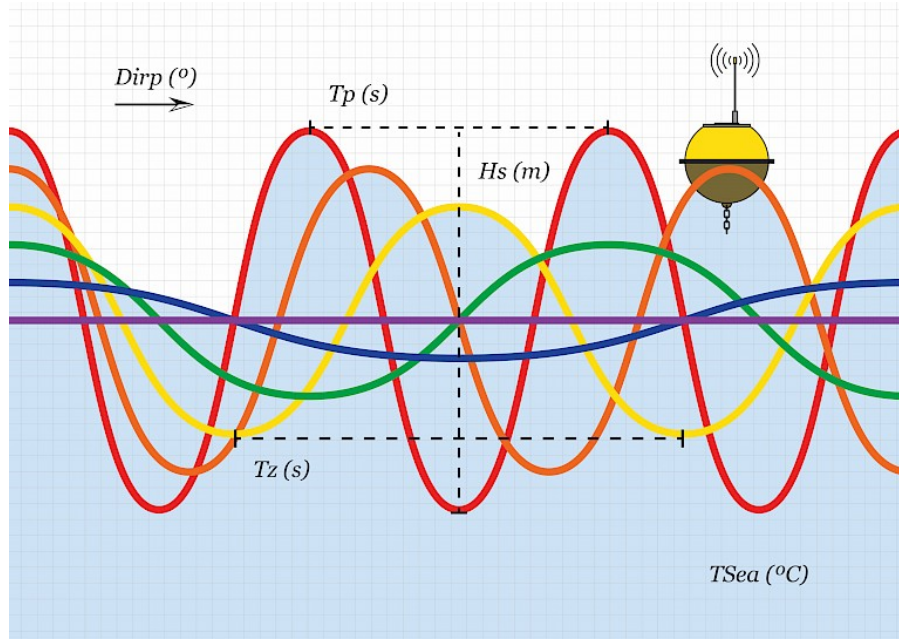


Figura 2.1: Representación visual de algunos parámetros característicos del oleaje, como H_s , T_p y T_z . Fuente: Folkestone Fringe.

2.2 Android como plataforma de sensado de bajo coste

El desarrollo de IOT tiene un amplio espectro de enfoques y variedad de desarrollos, desde soluciones de hardware dedicado, desarrollo de menos especializado y de código abierto, hasta soluciones comerciales cerradas. En este contexto, el uso de smartphones como plataforma de iot es una respuesta inteligente ya que si analizamos las características de los smartphones modernos, vemos que cuentan con una amplia variedad de sensores integrados, incluyendo acelerómetros, giroscopios y magnetómetros, que pueden ser utilizados para capturar datos inerciales sin necesidad de hardware adicional. Además, los smartphones ofrecen capacidades de procesamiento y conectividad que permiten no solo la captura de datos, sino también su procesamiento local y transmisión a servidores o nubes para análisis posterior.

Esto permite validar sistemas funcionales sin depender desde el primer momento de hardware específico, y esta reutilización de una plataforma de consumo encaja bien en arquitecturas IoT, en las que un mismo nodo puede asumir varias funciones de forma simultánea:

adquisición, preprocesado, visualización y comunicación, especialmente en arquitecturas de tipo edge.

Android aporta además de una base práctica para captura de medidas inerciales una plataforma que ayuda a acelerar la validación del sistema completo, ya que permite ahorrar tiempo de desarrollo en fases tempranas.

Frente a soluciones IoT dedicadas, Android permite iterar rápido y desplegar prototipos funcionales sin depender de hardware específico desde el primer día.



Figura 2.2: Referencia visual del smartphone como dispositivo de sensado, con integración de sensores y procesamiento local. Fuente: Medium.

2.2.1 Límites de una solución móvil pura

El enfoque móvil evidentemente presenta claras limitaciones como pueden ser:

- **Variabilidad entre dispositivos:** Existen innumerables modelos de smartphones en el mercado, cada uno con diferentes sensores, calibraciones y procesamiento; además de esto, existen diferentes versiones de Android, e incluso no tiene por qué ser Android. Teniendo esto en cuenta, no podemos esperar un comportamiento estándar, especialmente si se espera un nivel de precisión mínimamente profesional, por lo que es importante considerar esta variabilidad en el diseño del sistema y en la interpretación de los datos capturados.
- **Control limitado del hardware sensorial:** Los smartphones no están pensados para ser configurables a nivel de hardware, lo que implica que, aun teniendo acceso, en nuestro caso, a los sensores inerciales, no podemos controlar ciertas características como la frecuencia de muestreo, la sensibilidad, etc. Evidentemente, si comparamos esto con una solución de hardware dedicado, es una desventaja clara.

Si analizamos el rendimiento de los sensores inerciales entre dos smartphones, se aprecian diferencias por varias razones:

- **Diferente hardware físico:** Dos móviles pueden montar sensores MEMS distintos, ya sea por fabricantes, modelos o generaciones, lo que puede resultar en diferencias significativas en la calidad de la señal capturada.
- **Diferente sesgo (bias):** Aun con el mismo modelo de sensor, cada unidad puede tener un sesgo diferente, ya que se trata de un error sistemático. En el caso de un móvil que está quieto, el acelerómetro debería registrar una aceleración de 0 m/s^2 en los ejes horizontales y 9.81 m/s^2 en el eje vertical (debido a la gravedad). Pero cuando lo ponemos en práctica, debido a imperfecciones, puede dar un pequeño desplazamiento constante.
- **Diferente ruido (noise):** Además de ese desplazamiento constante, aun estando quietos, la señal no es perfectamente estable, sino que presenta pequeñas fluctuaciones aleatorias, y esta oscilación puede ser mayor o menor dependiendo del modelo de móvil, del estado de los sensores, etc.
- **Diferente tratamiento por el sistema operativo:** La app no accede al sensor físico directamente, sino a través del sistema operativo que muchas veces entrega la señal ya procesada, con filtros, ajustes, remuestreos o tratamientos de drivers y capas internas.
- **Diferente frecuencia real de muestreo:** Se puede pedir una frecuencia concreta, pero no siempre se cumple exactamente; puede tener jitter y variar según el modelo o la carga del sistema.
- **Diferente calibración:** Un teléfono puede venir con mejor calibración de fábrica que otro, lo que afecta a la exactitud, linealidad y estabilidad de la señal capturada.

2.3 Nodos embebidos para adquisición dedicada

Hay dos razones principales por las que se opta por un nodo embebido además de la solución móvil:

- **Comparación directa de captura.** La idea de usar nodos embebidos es obtener una comparación directa entre la captura que se obtiene con el dispositivo móvil y la captura que se obtiene con el nodo embebido. De esta forma se dispone de un punto de referencia para evaluar la calidad de la propuesta frente a una solución de hardware dedicado, que conllevaría un coste, complejidad y tiempo de desarrollo mayores.
- **Caso de uso extra.** Además de esto, disponer de más de un nodo de captura de datos abre la posibilidad de aplicar otras configuraciones. En este caso, el uso principal es la caracterización de oleaje en un punto concreto, pero si se tienen dos nodos podrían, por

ejemplo, colocarse en dos puntos de un barco y obtener datos de ambos lugares para realizar algún tipo de análisis adicional.

En este trabajo, esa línea se representa con ESP32+IMU, usando el ESP32 como nodo de captura/comunicación y Android como interfaz de operación. La comparación entre móvil y embebido no se entiende como una sustitución directa, sino como un análisis de compromisos entre simplicidad operativa, control técnico y mantenibilidad.

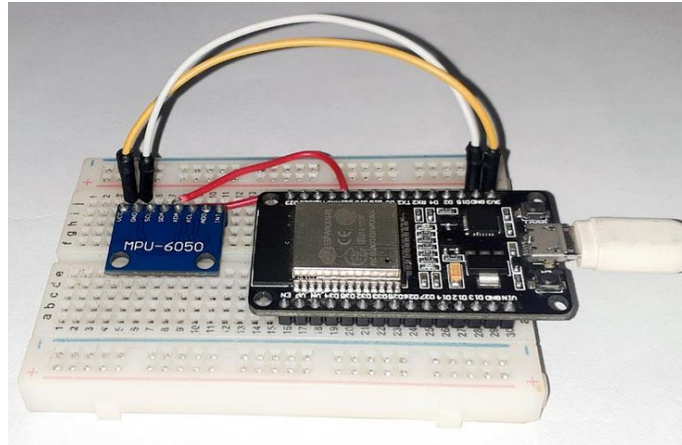


Figura 2.3: Referencia visual del subsistema embebido de captura, formado por una placa de control/comunicación y una IMU externa. Fuente: Pinterest.

2.4 Marco comercial y soluciones profesionales de referencia

Para situar adecuadamente la propuesta desarrollada en este TFG, conviene tener en cuenta que el mercado ya dispone de soluciones profesionales específicamente orientadas a la medida de oleaje y al sensado inercial de altas prestaciones. Estas soluciones no constituyen el objetivo directo del trabajo, pero sí ofrecen un marco de referencia útil para entender qué prestaciones, garantías y compromisos son habituales en el ámbito comercial.

En el caso de la medida de oleaje, existen boyas profesionales diseñadas específicamente para operación marina continuada, como las familias Waverider de Datawell. Este tipo de sistemas se orienta a la obtención fiable de parámetros estándar de oleaje, datos espectrales y operación prolongada en condiciones ambientales exigentes. Su interés dentro del estado del arte no reside únicamente en la precisión instrumental, sino también en el hecho de representar una solución consolidada, robusta y pensada desde el inicio para el entorno marino.

De forma complementaria, en el ámbito del sensado inercial existen IMUs industriales y profesionales concebidas para integración técnica en sistemas embebidos y de navegación, como ocurre con soluciones de fabricantes como VectorNav o Xsens. Frente a un smartphone, este tipo de sensores suele ofrecer especificaciones más estables, mejor caracterización

de sesgo y ruido, interfaces de integración más controladas y una mayor trazabilidad de las prestaciones declaradas por el fabricante. Esto los convierte en una referencia natural cuando se quiere comparar una solución accesible y flexible con instrumentación específicamente diseñada para tareas de medida o navegación.

Desde esta perspectiva, la propuesta del TFG no pretende competir directamente con una boya oceanográfica profesional ni sustituir una IMU industrial de altas prestaciones. Su interés se sitúa en otro plano: explorar una arquitectura de adquisición y procesado de bajo coste, flexible y extensible, capaz de integrar plataformas heterogéneas y de comparar distintos compromisos entre accesibilidad, control técnico y complejidad de despliegue.

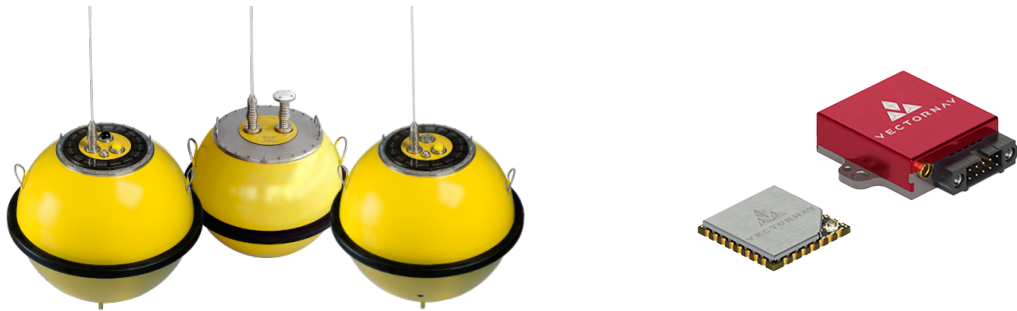


Figura 2.4: Ejemplos de soluciones profesionales de referencia en medida de oleaje y sensado inercial: familia de boyas Datawell Waverider (izquierda) y sensor VectorNav VN-200 (derecha). Fuentes: Datawell y VectorNav.

2.5 Posicionamiento del trabajo

La aportación del TFG se sitúa en la integración de arquitectura, no en un único componente aislado. El trabajo se posiciona como una propuesta de sistema ciberfísico modular que cubre adquisición, comunicación, persistencia y operación.

Aspecto	Smartphone	Nodo embebido propio	Solución profesional
Coste de entrada	Muy bajo, al reutilizar hardware de consumo	Moderado, con necesidad de integración y montaje	Alto, por tratarse de instrumentación específica
Control del sensor	Limitado por hardware y sistema operativo	Alto, al poder elegir interfaz, bus y configuración	Alto, con especificaciones y calibración declaradas
Robustez instrumental	Variable entre dispositivos	Dependiente del diseño e integración realizada	Alta, pensada para operación prolongada
Tiempo de despliegue	Muy rápido	Medio, requiere integración hardware/-software	Bajo una vez adquirido, pero con mayor barrera de acceso
Flexibilidad arquitectónica	Alta para prototipado y validación inicial	Alta, con mayor control técnico	Más limitada al producto adquirido
Objetivo principal	Accesibilidad y validación rápida	Adquisición dedicada y comparación técnica	Medida fiable y operación profesional

Cuadro 2.1: Comparación general entre smartphone, nodo embebido propio y soluciones profesionales de referencia.

Aspecto	Enfoques frecuentes	Enfoque del TFG
Plataforma base	App móvil o nodo embebido por separado	Arquitectura híbrida Android+ESP32
Persistencia	Almacenamiento local o backend ad-hoc	Persistencia desacoplada con alternativas evaluables
Integración	Piezas aisladas (captura, visualización, etc.)	Flujo extremo a extremo con contratos comunes
Evolución técnica	Cambios con alto acoplamiento	Diseño modular con sustitución de capas

Cuadro 2.2: Posicionamiento resumido del trabajo frente a enfoques habituales.

Análisis del problema, requisitos, planificación y presupuesto

EN este capítulo se analiza el problema que aborda el TFG, se fijan los requisitos del sistema y se recoge la planificación de trabajo seguida durante el desarrollo. También se incluyen los riesgos principales, los medios necesarios y una primera estimación económica del proyecto.

3.1 Análisis del problema

El objetivo del trabajo no es construir una instrumentación oceanográfica profesional desde la primera iteración, sino definir un sistema capaz de capturar, gestionar y explotar datos inerciales con una base operativa común para distintas plataformas de adquisición.

La formulación inicial del problema estuvo ligada a la posibilidad de reconstruir directamente la forma de cada ola a partir de medidas inerciales. Tras revisar las limitaciones reales de sensores de consumo y el tipo de información que se maneja habitualmente en este dominio, el problema se reformuló en términos más realistas: obtener parámetros útiles del estado del mar a partir de sesiones inerciales y permitir que esos datos puedan capturarse, almacenarse, consultarse y reprocesarse de forma homogénea.

Esta reformulación tiene dos consecuencias directas. La primera es que la captura deja de estar asociada a una única plataforma. La segunda es que el valor principal del sistema pasa a estar en la organización del flujo completo: adquisición, control, persistencia, comparación y explotación posterior.

Dentro de ese marco, el smartphone se adopta como punto de partida por motivos prácticos: integra sensores, capacidad de procesamiento, conectividad y una interfaz de operación completa. Sobre esa base se plantea la incorporación de un nodo embebido dedicado para comparar plataformas, ganar control técnico sobre la captura y preparar la evolución hacia

escenarios de operación remota y multi-nodo.

3.2 Requisitos del sistema

3.2.1 Requisitos funcionales

Los requisitos funcionales principales del sistema son los siguientes:

- capturar datos inerciales desde nodos de adquisición heterogéneos, incluyendo nodo móvil Android y nodo embebido ESP32+IMU;
- almacenar sesiones con metadatos trazables;
- visualizar estado de nodo, sesiones y resultados de forma operativa;
- exportar sesiones para análisis externo;
- soportar comandos básicos de control remoto, como inicio, parada, estado y configuración;
- permitir que la lógica de captura permanezca desacoplada de la interfaz, de modo que la operación pueda realizarse con la app en segundo plano o como consola de supervisión.

3.2.2 Requisitos no funcionales

Además de la funcionalidad visible, el sistema debe cumplir los siguientes requisitos no funcionales:

- modularidad entre adquisición, comunicación, backend y visualización;
- mantenibilidad para evolucionar backend o transporte sin rehacer el sistema;
- bajo coste de despliegue en entorno de TFG;
- trazabilidad de sesiones y de parámetros de ejecución;
- facilidad de operación en laboratorio y escenarios remotos;
- capacidad de escalar a escenarios con más de un nodo sin degradación operativa inaceptable.

3.2.3 Restricciones técnicas

Las restricciones técnicas más relevantes son:

- heterogeneidad de sensores y temporización en Android;
- limitaciones de recursos y estabilidad de integración en ESP32;
- necesidad de definir contratos de datos comunes entre nodos;
- tiempo de desarrollo limitado para cerrar integración extremo a extremo;
- incertidumbre sobre qué transporte remoto aporta mejor equilibrio en este alcance;
- necesidad de estimar experimentalmente la capacidad práctica del sistema en escenarios multi-nodo, en función de frecuencia de envío, tamaño de mensaje y latencia aceptable.

3.3 Casos de uso principales

Los casos de uso que estructuran el sistema son:

- configuración remota y consulta de estado de nodos de captura;
- inicio y parada remotos de adquisición sobre nodos móviles o embebidos;
- recuperación, consulta histórica y reprocesado de sesiones capturadas;
- comparación de sesiones entre nodos o dispositivos bajo un mismo flujo de explotación;
- supervisión de operación con la app actuando como interfaz de control, sin depender de interacción continua en primer plano.

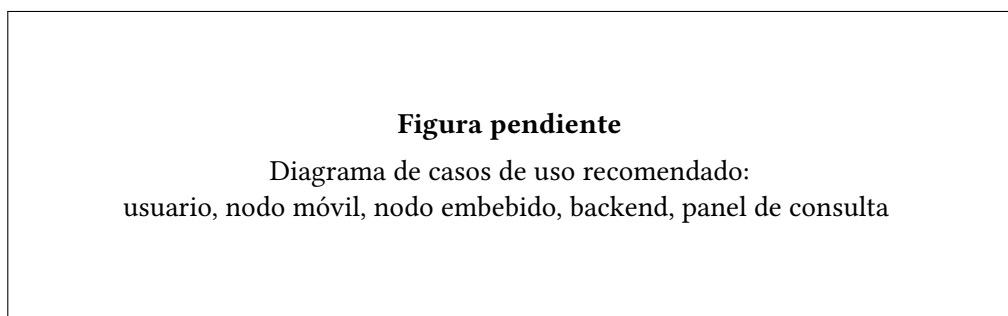


Figura 3.1: Casos de uso principales del sistema.

3.4 Metodología y planificación

3.4.1 Enfoque de trabajo

Se adopta una **metodología incremental** con un **diseño modular**. El trabajo parte de un análisis inicial del problema, del estado del arte y de las tecnologías disponibles para definir la solución. A partir de ahí, el desarrollo se organiza en iteraciones sucesivas en las que se implementa y valida un bloque funcional del sistema.

Este enfoque permite cerrar primero una base operativa del producto y reducir bloqueos por dependencias de infraestructura, integración o hardware. Además, facilita revisar decisiones con entregables intermedios y ajustar el alcance con el tutor conforme avanza el proyecto.

Durante el desarrollo se plantean reuniones periódicas con el tutor para revisar el avance, evaluar las decisiones tomadas y ajustar el trabajo en función de las indicaciones recibidas.

De forma resumida, la estrategia por bloques funcionales es:

1. captura y persistencia de sesión en local;
2. interfaz operativa Android y gestión de sesiones;
3. integración local con ESP32 vía interfaz de red;
4. incorporación progresiva de supervisión y acceso externo a datos;
5. evaluación final del sistema y propuestas de mejora.

3.4.2 Fases del proyecto

La planificación general del trabajo se descompone en las siguientes fases:

- **Fase 1.** Análisis de trabajos relacionados en sensores inerciales, análisis de movimiento y arquitecturas de captura de datos.
- **Fase 2.** Estudio de limitaciones y características de sensores IMU y técnicas de análisis aplicables.
- **Fase 3.** Definición de la solución y de la interacción entre sus módulos.
- **Fase 4.** Implementación de captura y gestión de datos en diferentes plataformas.
- **Fase 5.** Evaluación del sistema desarrollado y análisis de comportamiento, limitaciones y mejoras.
- **Fase 6.** Aplicación del sistema al caso práctico propuesto.
- **Fase 7.** Documentación final y presentación.

3.5 Riesgos y estrategia de validación

Como todo proyecto de desarrollo, el trabajo presenta riesgos técnicos que conviene identificar desde el principio para evitar bloqueos significativos. Los más relevantes son:

- **Desalineación entre nodos:** fijar un contrato de sesión común y validarlo en ambos modos de captura.
- **Inestabilidad de integración embebida:** mantener diagnósticos de conexión, bus y captura.
- **Acoplamiento excesivo a backend o transporte:** mantener desacopladas las capas de comunicación y persistencia.
- **Sobrecarga de alcance:** priorizar una base operativa funcional antes de añadir extensiones opcionales.

Desde el punto de vista de validación, el trabajo requiere comprobar no solo la captura aislada en cada plataforma, sino también la consistencia del flujo completo entre adquisición, sesión, consulta y explotación. En esta misma línea, la evolución hacia escenarios con más de un nodo hace recomendable medir de forma experimental la capacidad práctica de comunicación bajo diferentes tasas de envío y tamaños de mensaje.

3.6 Material y medios necesarios

Los medios necesarios para ejecutar el proyecto son:

- PC para desarrollo, pruebas y análisis de resultados;
- dispositivo móvil Android con sensores inerciales;
- plataforma embebida para captura de datos con conectividad inalámbrica;
- sensor IMU externo, con acelerómetro y giróscopo;
- herramientas de desarrollo software y entornos de programación;
- herramientas para almacenamiento, visualización y análisis de datos.

3.7 Presupuesto orientativo

Con el objetivo de reflejar de forma más realista el esfuerzo material del proyecto, se distinguen dos lecturas del presupuesto:

- **recursos utilizados por el proyecto**, incluyendo medios ya disponibles pero necesarios para desarrollar y validar el sistema;
- **adquisiciones imputables al proyecto**, centradas en el material específico comprado para la implementación.

Los valores de esta primera versión deben entenderse como una referencia preliminar pendiente de ajuste final con datos verificados.

3.7.1 Recursos utilizados por el proyecto

Concepto	Cantidad	Coste unitario	Coste estimado
PC de desarrollo y pruebas	1	900,00 €	900,00 €
Dispositivo Android con sensores inerciales	1	250,00 €	250,00 €
Placa ESP32 para nodo embebido	1	15,00 €	15,00 €
Sensor IMU MPU6050	1	5,00 €	5,00 €
Cableado, protoboard y pequeños consumibles	1	15,00 €	15,00 €
Herramientas software y entornos de desarrollo	1	0,00 €	0,00 €
Total estimado de recursos utilizados			1185,00 €

Cuadro 3.1: Primera estimación de los recursos utilizados por el proyecto, incluyendo medios ya disponibles.

3.7.2 Adquisiciones imputables al proyecto

Concepto	Cantidad	Coste unitario	Coste estimado
Placa ESP32 para integración embebida	1	15,00 €	15,00 €
Sensor IMU MPU6050	1	5,00 €	5,00 €
Cableado, conectores y protoboard	1	10,00 €	10,00 €
Consumibles de montaje y prueba	1	10,00 €	10,00 €
Total estimado de adquisiciones			40,00 €

Cuadro 3.2: Primera estimación del material comprado o imputable directamente al desarrollo del proyecto.

Incluso en esta primera aproximación se aprecia que el grueso del coste del TFG no reside en hardware especializado, sino en la integración, el desarrollo software, las campañas de prueba y la validación extremo a extremo del sistema.

Diseño de la plataforma

ESTE capítulo define la organización del sistema propuesto y fija las decisiones de diseño que permiten operar con dos plataformas de adquisición distintas dentro de un mismo flujo de trabajo. El diseño se apoya en una idea simple: separar adquisición, comunicación, persistencia y operación para poder sustituir componentes sin romper el resto del sistema.

4.1 Entorno y herramientas de desarrollo

Antes de definir la arquitectura funcional, resulta conveniente fijar las herramientas de trabajo previstas para el desarrollo, las pruebas y la documentación técnica del sistema. Dado que el proyecto integra aplicación móvil, nodo embebido, scripts de análisis y documentación, no se utiliza una única herramienta de programación, sino un conjunto de entornos adaptados a cada parte del sistema.

El desarrollo general del repositorio se realiza con **Visual Studio Code**, que se emplea como editor principal para la memoria en LaTeX, los scripts de apoyo en Python, la revisión de ficheros de datos y la organización del código del proyecto. Esta herramienta resulta adecuada para mantener en un mismo espacio de trabajo la documentación, los experimentos y los componentes software auxiliares.

Para la aplicación Android se utiliza el entorno propio del ecosistema Android, basado en **Android Studio**, **Gradle** y el SDK de Android. Android Studio permite compilar, desplegar y depurar la aplicación en el dispositivo móvil, además de revisar el comportamiento de la interfaz y el acceso a sensores. Aunque parte del código pueda editarse desde Visual Studio Code, la validación práctica de la app se apoya en las herramientas oficiales de Android.

En la parte embebida se plantea una evolución por fases. En las primeras pruebas con el ESP32 y la IMU resulta útil emplear **Python/MicroPython** desde **Thonny**, ya que permite comprobar de forma rápida la conectividad, el bus I2C, la lectura básica del sensor y pequeños prototipos de comunicación sin cerrar todavía una implementación final. Una vez verificado

el comportamiento básico del hardware, el desarrollo más fino del firmware se traslada a **C/C++**, usando el **IDE de Arduino** o una cadena equivalente de desarrollo para ESP32. Esta segunda fase permite mayor control sobre temporización, memoria, red, captura y generación de sesiones.

Para los scripts de análisis y pruebas de procesado se emplea **Python**, con librerías de cálculo y visualización para validar sesiones, comparar resultados y generar material de apoyo. Estos scripts no sustituyen al producto final, pero facilitan la exploración de parámetros, la depuración del pipeline y la preparación de resultados reproducibles para la memoria.

La capa de backend remoto queda como una línea de evolución. En esta fase no se fija todavía una tecnología definitiva, pero se consideran adecuadas alternativas de bajo coste y despliegue rápido como **Firebase** o **Supabase**, ya que ofrecen autenticación, base de datos, almacenamiento de ficheros y sincronización sin necesidad de mantener infraestructura propia. La elección final debe hacerse atendiendo a criterios de simplicidad, coste, trazabilidad de sesiones, facilidad de integración con Android y capacidad de evolucionar hacia escenarios con varios nodos.

Para los diagramas de diseño se utiliza una combinación de herramientas. Los primeros borradores se pueden definir con **Mermaid**, por su facilidad para versionar diagramas junto al código. Para las figuras finales de la memoria se pueden emplear herramientas gráficas como **diagrams.net** o figuras generadas específicamente para el documento, exportadas como imagen e integradas en LaTeX. En los casos donde convenga mantener el diagrama directamente dentro de la memoria, también puede utilizarse **TikZ**.

4.2 Vista general del sistema

La plataforma se organiza en cuatro bloques funcionales:

- **adquisición**: captura de muestras y construcción de sesiones a partir de Android o ESP32;
- **comunicación**: intercambio de comandos de control y transferencia de datos entre nodos;
- **backend/persistencia**: almacenamiento local y posible sincronización remota;
- **interfaz/control**: operación del sistema, consulta de sesiones y visualización de resultados.

Sobre esta base, el sistema contempla dos modos operativos:

- **modo móvil**: el propio dispositivo Android actúa como nodo de captura, almacenamiento y explotación local;

- **modo embebido:** el nodo ESP32 realiza la captura y Android actúa como consola de supervisión, control y descarga.

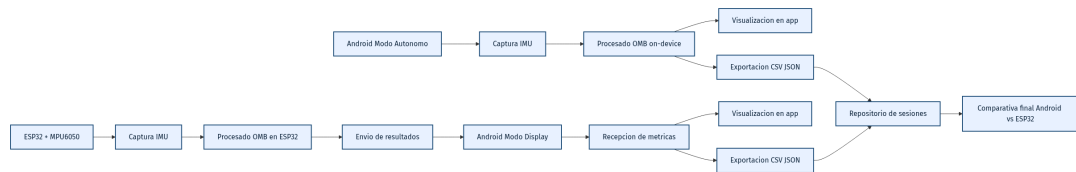


Figura 4.1: Arquitectura funcional del sistema con Android como plataforma central de operación.

La decisión de mantener ambos modos dentro de un mismo diseño evita desarrollar dos soluciones independientes. El objetivo no es tener una aplicación móvil por un lado y un nodo embebido por otro, sino un único sistema capaz de operar con ambos.

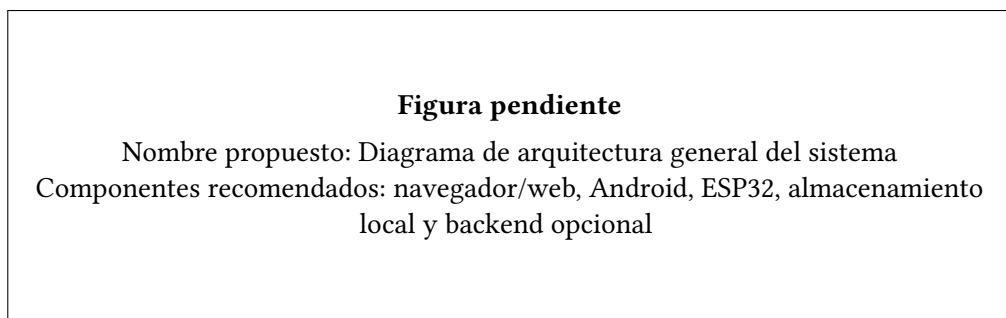


Figura 4.2: Hueco para diagrama de arquitectura general del sistema.

4.3 Diseño de adquisición

La adquisición es el punto de entrada del sistema. Independientemente del origen de datos, toda captura debe poder convertirse en una sesión homogénea, de forma que el resto de la plataforma no dependa directamente del hardware que ha producido las muestras.

4.3.1 Sesión como unidad de captura

La sesión es la unidad principal de intercambio entre subsistemas. Toda captura, independientemente de su origen, debe quedar representada con una estructura común que permita almacenamiento, comparación y reprocesado.

El contrato de sesión incluye, como mínimo:

- `device_id`;

- `source` (móvil/embebido/simulado);
- `timestamps`;
- `fs` estimada;
- `payload` de muestras inerciales;
- metadatos de calidad y contexto.

Esta decisión permite que el procesado y la explotación trabajen con una entrada homogénea aunque el origen de datos cambie.

4.3.2 Adquisición en Android

En el modo móvil, el propio dispositivo Android actúa como nodo de captura. Registra las muestras del sensor, construye la sesión y permite una revisión inmediata del resultado desde la propia aplicación. Este modo constituye la base operativa del sistema porque reduce dependencia de hardware adicional y acelera la validación funcional.

La operación mínima en este modo incluye:

- inicio y parada de captura;
- almacenamiento local de la sesión;
- consulta posterior y exportación.

4.3.3 Adquisición en ESP32

En el modo embebido, el nodo ESP32 se encarga de la captura y Android actúa como consola de operación. La sesión sigue siendo la misma unidad lógica de trabajo, aunque en este caso se genera fuera del teléfono y se recupera después a través de la capa de comunicación.

El subsistema embebido se compone de ESP32, IMU y electrónica auxiliar de conexión y alimentación. Su función es doble:

- actuar como nodo de adquisición dedicado;
- servir como base de validación para la interoperabilidad con la app Android.

El diseño embebido debe garantizar, como mínimo:

- captura de muestras y cierre de sesión;
- exposición de estado operativo;

- recepción de comandos de control;
- descarga posterior de la sesión generada.

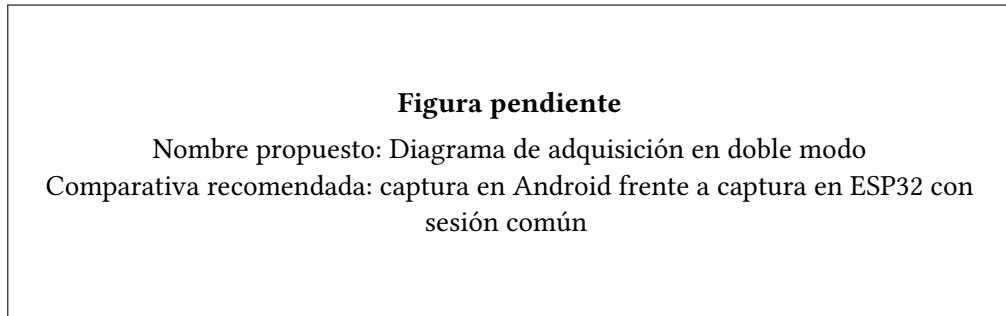


Figura 4.3: Hueco para diagrama de adquisición en modo móvil y modo embebido.

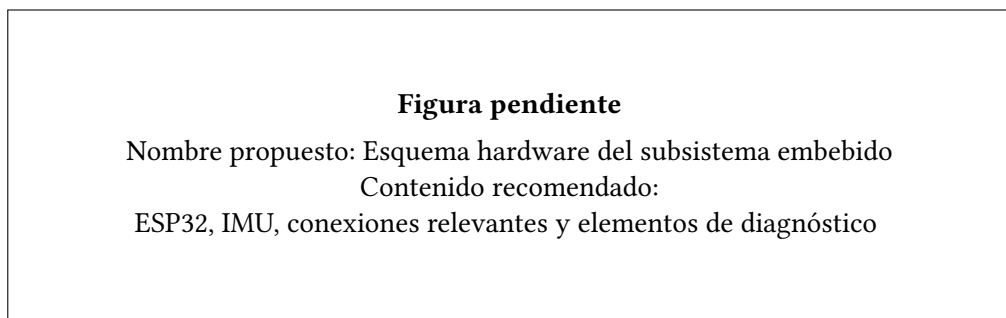


Figura 4.4: Esquema del subsistema embebido y conexiones principales.

4.3.4 Separación entre captura y procesado

El procesado se mantiene fuera de la capa de adquisición. Esta separación permite sustituir métodos de análisis sin alterar captura, comunicación o persistencia.

Cualquier bloque de procesado debe cumplir tres condiciones:

- entrada basada en contrato de sesión;
- salida estructurada y trazable;
- compatibilidad con ambos orígenes de datos.

Sobre esa base, el sistema puede acomodar tanto métodos espectrales basados en PSD y momentos como otros enfoques apoyados en IMUs de baja precisión, transformación de aceleraciones al eje vertical, técnicas de denoising e integración en frecuencia.

4.4 Diseño de comunicación

La comunicación entre componentes debe permitir que la operación del sistema no quede ligada a una única plataforma de captura. En la práctica, esto implica que Android pueda actuar como nodo de sensado o como consola de control, manteniendo un flujo operativo comparable en ambos casos.

4.4.1 Contrato de control

Para la operación remota y la coordinación de nodos se define un contrato mínimo de control:

- `start`;
- `stop`;
- `set_config`;
- `status`.

El contrato de control cubre lo estrictamente necesario para iniciar y detener capturas, consultar estado y modificar configuración sin acoplar el diseño a una implementación concreta de transporte.

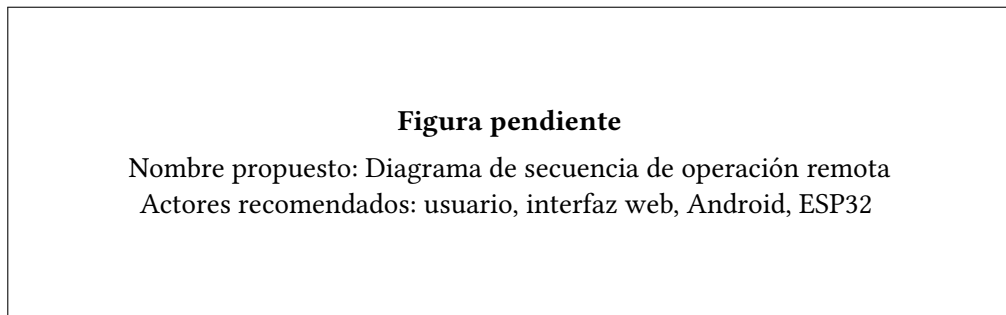


Figura 4.5: Hueco para diagrama de secuencia de operación remota.

4.4.2 Enlace entre Android y ESP32

La capa de comunicación cubre dos escenarios distintos:

- **enlace local Android-ESP32**, utilizado en laboratorio y en pruebas de integración;
- **sincronización remota con backend**, prevista para consulta y operación desacoplada.

El sistema se diseña para no depender desde el principio de una única tecnología de transporte. La interfaz entre nodos debe mantenerse estable aunque el mecanismo concreto evolucione.

Por este motivo, la elección final del mecanismo de comunicación entre Android y ESP32 no se fija únicamente por conveniencia de implementación. Debe apoyarse en una evaluación experimental de latencia, robustez, simplicidad de integración y comportamiento bajo concurrencia, de forma que la decisión técnica quede respaldada por medidas y no solo por preferencia de desarrollo.

4.4.3 Flujo de datos del sistema

El flujo de datos completo se define en cuatro pasos:

1. adquisición de muestras;
2. normalización y empaquetado de sesión;
3. persistencia local y/o sincronización remota;
4. recuperación para visualización, comparación y exportación.

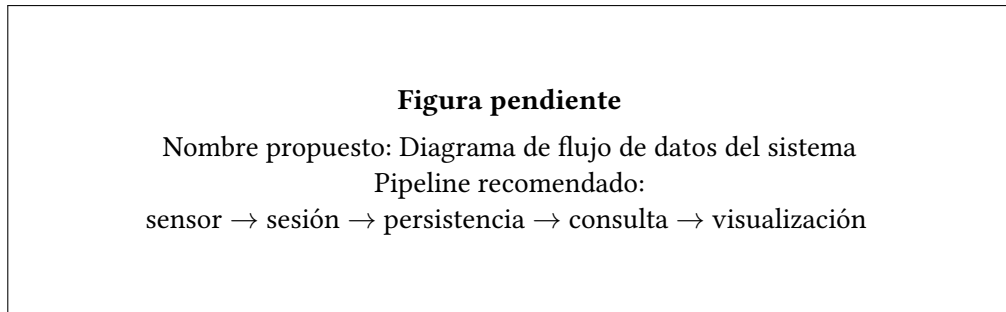


Figura 4.6: Flujo de datos extremo a extremo del sistema.

Este flujo es el mismo independientemente del origen de captura. La diferencia entre modo móvil y modo embebido no está en la explotación final, sino en quién produce la sesión y cómo se llega hasta ella.

4.5 Diseño de persistencia y backend

La persistencia se diseña con dos niveles complementarios:

- **persistencia local**, necesaria para que cada nodo pueda cerrar sesiones de forma autónoma;

- **persistencia remota**, necesaria cuando se quiera sincronizar sesiones, estado de nodos o información de operación.

El backend se plantea como una capa intercambiable. Sus capacidades mínimas son:

- persistencia de metadatos de sesión;
- almacenamiento de ficheros o exportaciones;
- sincronización de estado entre nodos.

La capa de explotación proporciona operaciones homogéneas de:

- listado de sesiones;
- detalle de sesión;
- exportación de datos.

Este contrato permite que la app trate sesiones locales, embebidas o sincronizadas bajo el mismo flujo de trabajo. La selección tecnológica final de backend se deja abierta y se decidirá por criterios de validación práctica: robustez de despliegue, mantenibilidad, coste y capacidad de escalar hacia escenarios con más de un nodo.

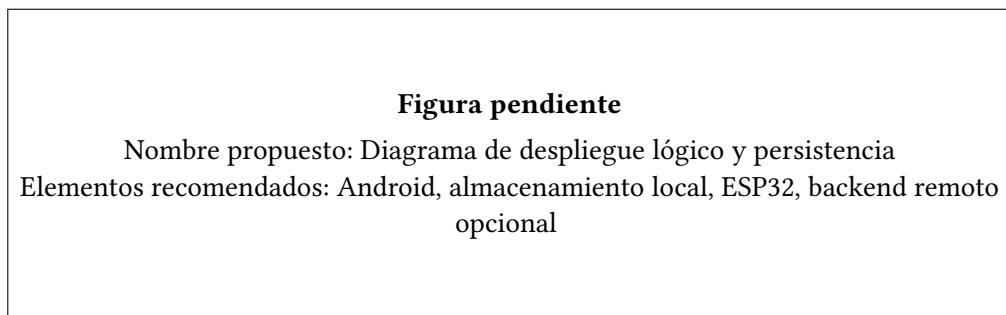


Figura 4.7: Hueco para diagrama de despliegue lógico y persistencia.

4.6 Diseño de interfaz y control

Android no se limita a la interfaz gráfica. Dentro del diseño propuesto, la aplicación asume tres funciones:

- nodo de captura cuando se trabaja en modo móvil;
- consola de operación cuando se trabaja con el subsistema embebido;

- punto de acceso a sesiones, resultados y exportación.

En consecuencia, la interfaz debe soportar dos flujos principales: operación local sobre la propia captura del teléfono y operación remota sobre el nodo ESP32. En ambos casos, el objetivo es mantener una experiencia homogénea de inicio de captura, consulta de estado, recuperación de sesión y revisión de resultados.

La comparativa entre plataformas forma parte de este diseño de interfaz y control. Para que la comparación tenga sentido, ambos modos deben compartir una estructura común de sesión, operaciones equivalentes de control y un flujo homogéneo de explotación.

Los ejes principales de comparación son estabilidad de captura, trazabilidad de sesión, latencia operativa y robustez de integración. La intención no es comparar únicamente valores numéricos, sino también el coste operativo y el grado de control técnico que aporta cada plataforma.

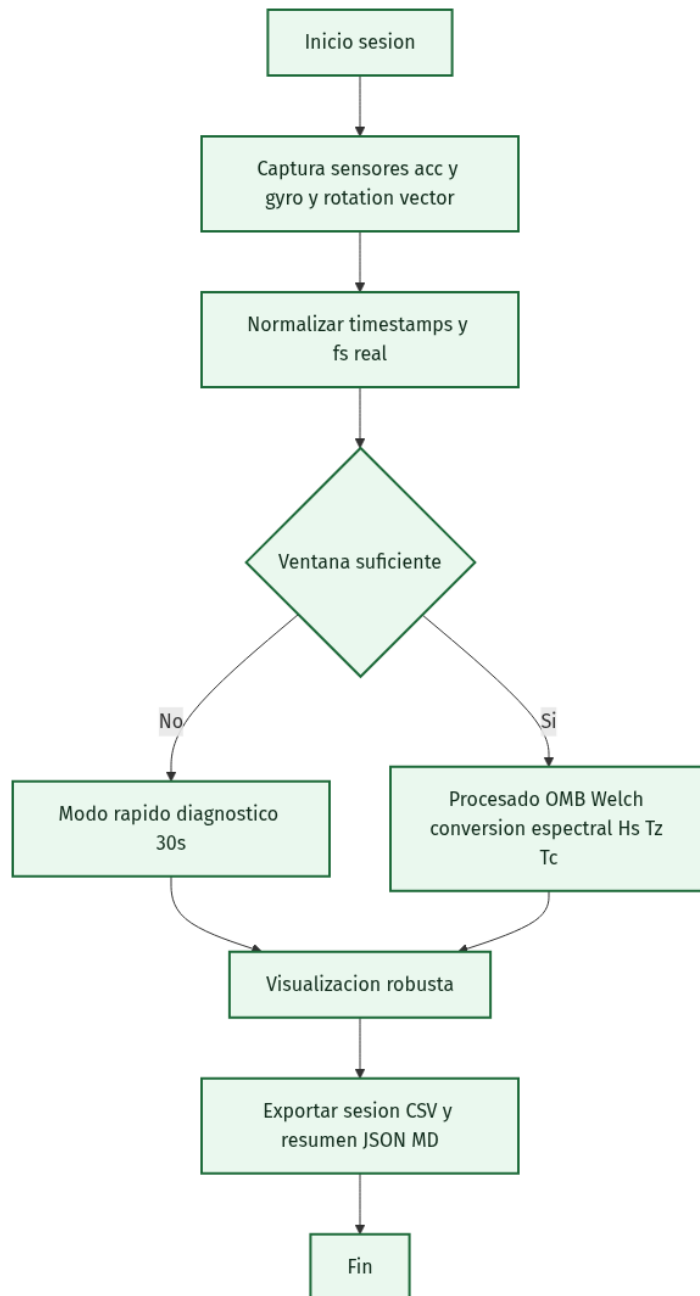


Figura 4.8: Flujo Android: captura/operación, gestión de sesiones y visualización.

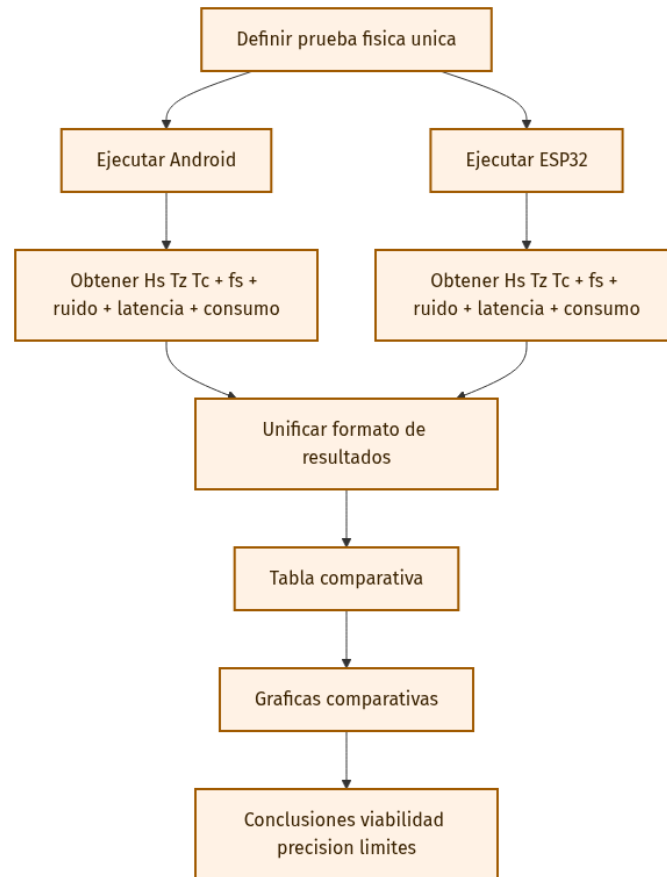


Figura 4.9: Flujo de comparativa entre nodo móvil y nodo embebido con contratos comunes.

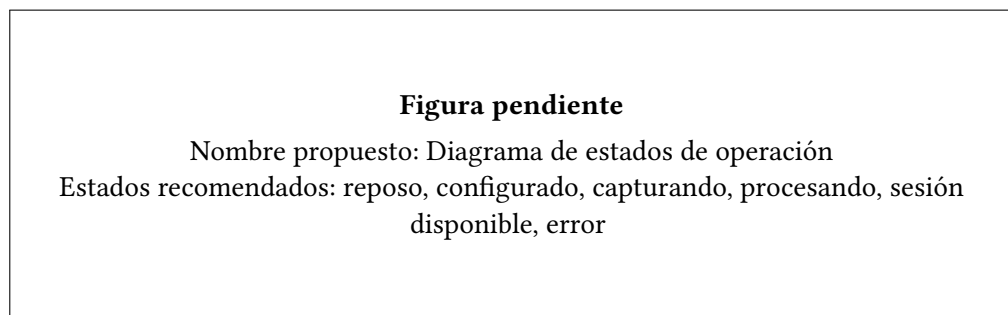


Figura 4.10: Hueco para diagrama de estados de operación del sistema.

Implementación del sistema

EN este capítulo se documenta la implementación del sistema siguiendo la metodología incremental descrita en capítulos anteriores. A diferencia del diseño, que define la arquitectura objetivo, la implementación se presenta como una evolución por fases. Esta forma de organizar el capítulo permite reflejar con claridad qué partes están ya desarrolladas, cuáles se encuentran en prueba y cuáles quedan como trabajo pendiente.

5.1 Criterio de implementación incremental

El desarrollo del sistema se plantea de forma progresiva para reducir el riesgo técnico. En lugar de intentar cerrar desde el principio una solución completa con aplicación móvil, nodo embebido, procesado y backend, se implementan primero bloques pequeños que puedan probarse por separado.

El orden de trabajo seguido es el siguiente:

1. preparación del entorno de desarrollo y estructura del repositorio;
2. pruebas iniciales de captura y procesado con scripts de apoyo;
3. implementación del modo móvil en Android;
4. pruebas de lectura de IMU en ESP32 mediante prototipos sencillos;
5. migración del nodo embebido a una implementación más controlada en C/C++;
6. definición de un formato común de sesión y resultados;
7. integración progresiva entre Android, ESP32 y, posteriormente, backend remoto.

Esta estrategia permite validar cada decisión antes de acoplarla al resto del sistema. Además, facilita mantener trazabilidad entre pruebas, código, sesiones capturadas y resultados obtenidos.

5.2 Estado de partida de la implementación

El punto de partida del trabajo práctico es un repositorio único que agrupa la memoria, los scripts de experimentación, la aplicación Android, el firmware del nodo embebido y los datos de prueba. Esta organización simplifica la continuidad del proyecto, ya que permite mantener juntos código, documentación y resultados.

En esta fase inicial se distinguen tres líneas de trabajo:

- **línea Android:** aplicación móvil para captura, gestión local y visualización;
- **línea ESP32:** pruebas de hardware, lectura de IMU y comunicación con la app;
- **línea de análisis:** scripts externos para validar el procesado antes de integrarlo definitivamente en la aplicación.

La capa de backend remoto no se considera todavía parte implementada del sistema. Se mantiene como una línea de evolución que se incorporará cuando el flujo local y el formato de datos estén suficientemente estabilizados.

5.3 Herramientas y organización del código

La implementación se organiza según las herramientas descritas en el Capítulo 4. Visual Studio Code se utiliza como entorno general para revisar el repositorio, editar la memoria, mantener scripts y consultar resultados. Android Studio se emplea para compilar, desplegar y depurar la aplicación Android.

Para el nodo embebido se contempla una primera fase de pruebas con Python y MicroPython en Thonny, orientada a verificar la comunicación con la IMU y el comportamiento básico del hardware. Tras esa fase, la implementación principal del firmware se desarrolla en C/C++, con el objetivo de tener mayor control sobre temporización, memoria, red y generación de sesiones.

La estructura del repositorio separa los componentes principales:

- `android_app_boya/`: aplicación Android;
- `esp32/`: código del nodo embebido;
- scripts Python en la raíz del repositorio: captura, procesado y diagnóstico;
- `data/`: sesiones y resultados de pruebas;
- `modelo-tfg-fic-v1.6_2223xun/`: memoria del TFG.

5.4 Fase 1: scripts de captura y análisis

Antes de cerrar la funcionalidad dentro de Android, se utilizan scripts en Python para explorar el problema de captura y procesamiento de datos inerciales. Esta fase permite trabajar con ciclos de prueba rápidos y observar el comportamiento de la señal sin depender de una interfaz móvil completa.

Los scripts de apoyo cumplen tres funciones:

- recibir o leer sesiones de datos IMU;
- aplicar diferentes configuraciones de procesamiento;
- generar métricas y gráficas para entender la respuesta de la señal.

Esta fase también sirve para corregir el enfoque inicial del proyecto. Las primeras pruebas muestran que la doble integración de aceleración para obtener una altura instantánea es muy sensible a ruido, orientación y deriva. Por ello, el procesamiento se orienta hacia una estimación espectral por ventana, siguiendo un enfoque tipo OpenMetBuoy, con cálculo de parámetros agregados como H_s , T_z , T_c y T_p .

[AÑADIDO]

Pendiente de completar con resultados concretos:

- listar scripts finalmente usados;
- indicar una o dos sesiones representativas;
- añadir una figura del pipeline de procesamiento;
- resumir qué decisión técnica se tomó a partir de estas pruebas.

5.5 Fase 2: implementación del modo móvil en Android

El modo móvil es la primera implementación funcional de la aplicación Android. En esta fase no se persigue todavía cerrar la experiencia final de usuario, sino construir una base práctica que permita comprobar que la captura de sensores, el guardado de sesiones y la navegación entre pantallas funcionan de forma controlada.

Por este motivo se desarrolla una interfaz sencilla de depuración. Esta interfaz sirve para lanzar capturas manuales, listar los ficheros generados, revisar información básica de cada sesión y preparar puntos de entrada para fases posteriores. El objetivo operativo a medio plazo es que la captura pueda ejecutarse en segundo plano y que la interfaz quede reducida a configuración, diagnóstico y consulta puntual.

5.5.1 Estructura general de la aplicación

La aplicación se implementa como un proyecto Android nativo en Java, con Gradle como sistema de construcción. La estructura inicial separa la navegación y las responsabilidades principales en varias actividades:

- `MenuActivity`, como pantalla inicial de selección entre modo móvil y modo ESP32;
- `MobileModeActivity`, como pantalla de captura y gestión local de sesiones;
- `Esp32ModeActivity`, preparada como consola de diagnóstico e integración con el nodo embebido;
- `SessionDetailActivity`, reservada para la consulta detallada de sesiones en fases posteriores.

Esta separación permite validar primero el flujo móvil sin bloquear la evolución hacia el modo embebido. También evita concentrar toda la lógica en una única pantalla, algo importante para mantener la aplicación extensible conforme se incorporen captura en segundo plano, diagnóstico de comunicación y explotación de sesiones.

5.5.2 Captura de sensores

La captura usa las APIs nativas de sensores de Android, mediante el gestor de sensores del sistema y un listener de eventos. La actividad de modo móvil obtiene el acelerómetro y el giroscopio del dispositivo y registra muestras durante una ventana temporal configurable. Para cada muestra se guarda un tiempo relativo y las componentes disponibles de aceleración y giro.

El uso de `SystemClock.elapsedRealtimeNanos()` permite construir una referencia temporal relativa al inicio de la captura. Durante la adquisición se mantiene una lista de objetos `ImuSample`, que agrupan los campos `t_ms`, `ax`, `ay`, `az`, `gx`, `gy` y `gz`. La interfaz actual muestra el estado de la captura, el tiempo restante y el número de muestras acumuladas, lo que resulta útil para detectar problemas de frecuencia, ausencia de sensores o interrupciones durante las primeras pruebas.

En esta fase la captura se controla manualmente desde la pantalla de depuración. No obstante, la lógica se organiza de forma que pueda desplazarse posteriormente a un servicio o componente en segundo plano, manteniendo la interfaz como capa de configuración y supervisión.

5.5.3 Persistencia local de sesiones

Las sesiones se almacenan localmente mediante una clase específica del repositorio de datos: `SessionRepository`. Esta clase encapsula las operaciones sobre ficheros, crea una carpeta de sesiones dentro del espacio externo privado de la aplicación y proporciona métodos para guardar, listar, renombrar, eliminar y leer sesiones.

El formato inicial elegido es CSV, con cabecera:

`t_ms, ax, ay, az, gx, gy, gz`

Este formato se adopta por simplicidad, porque puede inspeccionarse fácilmente fuera de la aplicación y porque facilita comparar las sesiones generadas por Android con datos procedentes de scripts o del nodo embebido. Los nombres de fichero incluyen una marca temporal y la duración de la captura; si el usuario introduce un nombre manual, se normaliza para evitar caracteres problemáticos en el sistema de ficheros.

El Código 5.1 muestra de forma simplificada la escritura de una sesión. El aspecto más relevante de este fragmento es que la captura deja de ser una lectura puntual de sensores y pasa a convertirse en un artefacto persistente, con una estructura de columnas estable y reutilizable por el resto del sistema.

```

1 try (BufferedWriter writer = new BufferedWriter(new
  FileWriter(outFile))) {
2     writer.write("t_ms,ax,ay,az,gx,gy,gz\n");
3
4     for (ImuSample s : samples) {
5         writer.write(String.format(
6             Locale.US,
7             "%d,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f\n",
8             s.tMs, s.ax, s.ay, s.az, s.gx, s.gy, s.gz
9         ));
10    }
11
12    writer.flush();
13 }
```

Listing 5.1: Fragmento simplificado del guardado de una sesión IMU en formato CSV.

5.5.4 Gestión local y operaciones de depuración

La pantalla de modo móvil no se diseña como interfaz definitiva, sino como herramienta de trabajo para comprobar el comportamiento de la aplicación durante el desarrollo. Desde ella se puede seleccionar la duración de captura, asignar un nombre opcional a la sesión, iniciar o detener la adquisición y actualizar el listado de ficheros guardados.

- La pulsación normal sobre una sesión permite seleccionarla para pruebas posteriores.
- La pulsación larga abre un menú contextual con operaciones auxiliares, como consultar información del fichero, renombrarlo o eliminarlo.
- La lista de sesiones se actualiza desde el repositorio local, ordenando los ficheros por fecha de modificación.

Esta gestión local permite verificar rápidamente que las capturas se están generando, que el tamaño de los ficheros es coherente y que las sesiones pueden recuperarse en ejecuciones posteriores de la aplicación. En una versión más madura, estas operaciones deberían quedar integradas en una pantalla de ajustes, diagnóstico o administración de sesiones, mientras que la captura principal se ejecutaría sin depender de la interacción continua con la interfaz.

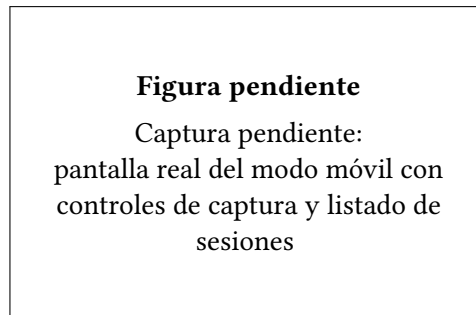


Figura 5.1: Pantalla real de la aplicación Android utilizada durante la depuración del modo móvil.

5.5.5 Prototipo de interfaz auxiliar

Antes de implementar una operación en segundo plano, se definió un prototipo visual de la aplicación para ordenar las pantallas necesarias durante las pruebas. La Figura 5.2 muestra esta idea: una pantalla inicial de selección de modo, una pantalla de control para el modo Android, una pantalla equivalente para el modo ESP32 y una vista de comprobación prevista para la revisión posterior de sesiones.

La existencia de estas pantallas no implica que la interfaz sea el objetivo principal del sistema. Su función en esta fase es reducir el tiempo de depuración y hacer visibles los estados intermedios de captura, persistencia y conexión. La evolución prevista consiste en desplazar la adquisición a un funcionamiento en segundo plano y conservar únicamente controles de configuración, estado y diagnóstico.

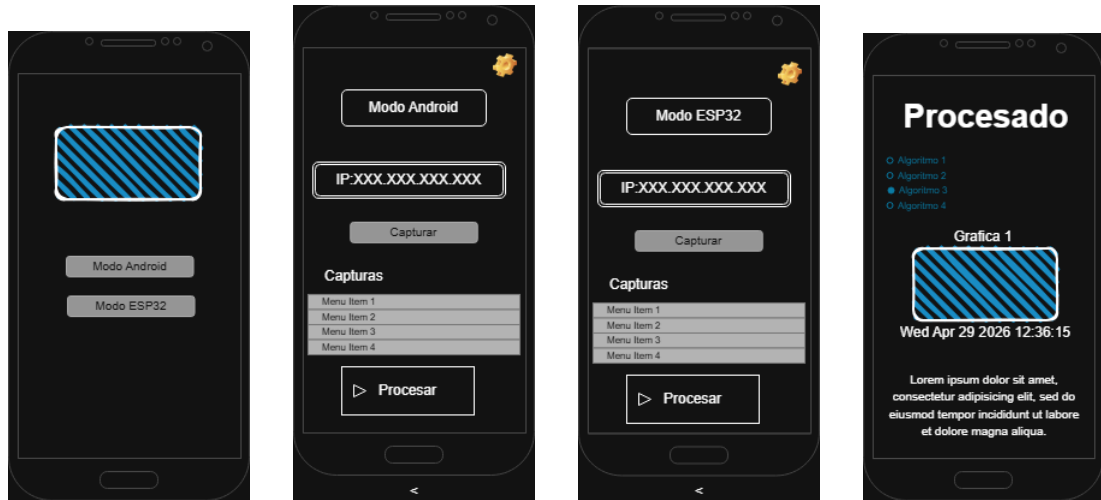


Figura 5.2: Prototipo de interfaz auxiliar usado para organizar las primeras pruebas de captura, gestión de sesiones y diagnóstico de modos de operación.

5.6 Fase 3: *procesado on-device*

Una vez cerrada la captura local, el siguiente paso es integrar el procesado dentro de Android. El algoritmo no se trata como un bloque aislado, sino como una parte de la aplicación que consume sesiones y devuelve resultados trazables.

El enfoque principal de procesado se basa en:

- estimar la frecuencia efectiva de muestreo;
- remuestrear o normalizar temporalmente la señal cuando sea necesario;
- calcular una estimación espectral mediante Welch;
- convertir el espectro de aceleración vertical a espectro de elevación;
- obtener métricas de oleaje mediante momentos espectrales.

Las métricas de salida previstas son H_s , T_z , T_c y T_p . Se debe mantener visible que estos valores representan una estimación por ventana, no una reconstrucción exacta de la altura instantánea de cada ola.

[AÑADIDO]

Pendiente de completar:

- explicar la clase o módulo concreto que implementa el procesado;
- indicar modos de procesado disponibles;

- añadir tabla de parámetros usados;
- comparar resultados Android frente a los scripts Python.

5.7 Fase 4: pruebas iniciales con ESP32 e IMU

La línea embebida se introduce después de tener una base funcional en Android. Su objetivo es disponer de un nodo de captura dedicado que permita comparar la solución móvil con una plataforma de mayor control técnico.

La primera fase con ESP32 se orienta a diagnóstico:

- comprobar alimentación y conexión del sensor;
- verificar el bus I2C;
- leer registros básicos de la IMU;
- obtener muestras de aceleración y giro;
- detectar problemas de cableado, dirección del dispositivo o estabilidad.

Para estas pruebas iniciales se puede utilizar MicroPython desde Thonny, ya que permite ejecutar código sencillo de forma interactiva y comprobar rápidamente si el hardware responde. Esta fase no pretende ser el firmware definitivo, sino una forma de reducir incertidumbre antes de pasar a una implementación más controlada.

[AÑADIDO]

Pendiente de completar:

- indicar modelo exacto de placa ESP32 usada;
- documentar conexiones SDA/SCL, alimentación y dirección I2C;
- añadir resultado de detección de la IMU;
- recoger incidencias encontradas durante las pruebas.

5.8 Fase 5: firmware en C/C++ para ESP32

Tras validar el hardware con pruebas simples, el nodo embebido debe implementarse en C/C++ para ganar control sobre temporización, memoria y comunicación. Esta fase busca convertir el ESP32 en un nodo operativo capaz de producir sesiones compatibles con el resto del sistema.

Las responsabilidades previstas del firmware son:

- inicializar la IMU;
- capturar muestras con una frecuencia definida;
- mantener estado de captura;
- exponer un mecanismo de comunicación local;
- generar una sesión descargable o enviar resultados estructurados.

La comunicación con Android se plantea inicialmente en red local, mediante un conjunto reducido de operaciones: comprobar estado, iniciar captura, detener captura y recuperar datos. Estas operaciones deben mantenerse simples para facilitar depuración y posterior comparación con otros mecanismos de comunicación.

[AÑADIDO]

Pendiente de completar:

- describir endpoints o comandos finalmente implementados;
- indicar cómo se almacena o genera la sesión en ESP32;
- medir frecuencia real de captura;
- documentar limitaciones de memoria y duración máxima de sesión.

5.9 Fase 6: formato común de sesión y resultados

Para que Android y ESP32 puedan compararse de forma justa, ambos modos deben producir datos bajo un contrato común. Esta parte de la implementación es importante porque evita que cada plataforma tenga su propio formato incompatible.

El formato común debe incluir:

- identificador de dispositivo u origen;
- marca temporal o eje temporal relativo;
- frecuencia estimada o configurada;
- muestras inerciales disponibles;
- parámetros de procesado;
- resultados calculados y unidades.

En una primera iteración se mantiene CSV para las muestras por su simplicidad. Para resultados y metadatos se prevé añadir un resumen estructurado en JSON, de modo que cada sesión pueda reproducirse y compararse sin depender únicamente del nombre del fichero.

[AÑADIDO]

Pendiente de completar:

- fijar nombres definitivos de columnas;
- definir campos del resumen JSON;
- incluir ejemplo mínimo de sesión;
- comprobar compatibilidad entre sesiones Android y ESP32.

5.10 Fase 7: backend remoto

La integración con backend remoto no se considera cerrada en el estado actual. Su papel dentro del sistema será sincronizar sesiones, resultados y estado de nodos cuando el flujo local sea estable.

Antes de implementar esta capa se deben resolver dos cuestiones:

- qué datos deben sincronizarse y con qué granularidad;
- qué tecnología ofrece mejor equilibrio entre sencillez, coste y mantenibilidad.

Como alternativas iniciales se consideran plataformas como Firebase o Supabase, por su integración rápida con aplicaciones móviles y por ofrecer base de datos, autenticación y almacenamiento de ficheros. La elección final deberá justificarse con criterios prácticos y no solo por familiaridad con la herramienta.

[AÑADIDO]

Pendiente de completar:

- comparar alternativas de backend;
- definir modelo mínimo de datos remoto;
- decidir si se sincronizan sesiones completas, resúmenes o ambos;
- implementar una prueba mínima de subida y consulta.

5.11 Criterios de cierre de implementación

Se considerará que la implementación queda suficientemente cerrada para el alcance del TFG cuando se cumplan los siguientes puntos:

- la app Android capture y guarde sesiones de forma repetible;
- el procesado on-device genere métricas trazables por sesión;
- el nodo ESP32 pueda capturar o generar sesiones compatibles con el formato común;
- exista una prueba de integración entre Android y ESP32;
- se documenten claramente las partes implementadas y las partes pendientes;
- los resultados puedan reproducirse a partir de sesiones guardadas.

Este cierre no implica que el sistema sea una boya oceanográfica profesional, sino que la arquitectura propuesta se ha llevado a una implementación funcional y evaluable dentro del alcance de un TFG.

Validación y evaluación

LA validación se orienta a comprobar la viabilidad del sistema como arquitectura operativa y a medir el grado de madurez de integración entre sus capas.

6.1 Metodología de validación

La evaluación se diseña con enfoque de ingeniería aplicada, separando tres niveles:

- **validación funcional:** cada bloque cumple su responsabilidad;
- **validación de integración:** interoperabilidad entre Android, ESP32 y repositorio de sesiones;
- **validación operativa:** estabilidad del flujo de trabajo en escenarios de uso repetido.

El objetivo no es obtener un único resultado numérico, sino demostrar que el sistema funciona de forma trazable y mantenible.

6.2 Escenarios de prueba

Los escenarios principales considerados son:

- captura y gestión de sesión en modo móvil;
- operación remota local sobre ESP32 (estado, inicio/parada y descarga);
- consistencia del contrato de sesión entre orígenes móvil y embebido;
- robustez de operación tras múltiples iteraciones de captura y consulta;
- ensayo preliminar de capacidad de comunicación en escenario multi-nodo, para estimar el rango operativo del sistema ante concurrencia de varios emisores.

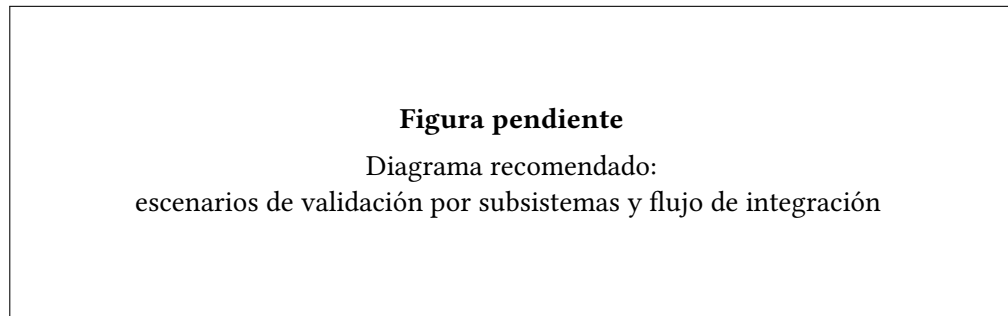


Figura 6.1: Escenarios de prueba del sistema.

6.3 Configuración de pruebas

Para cada prueba se documentan, como mínimo:

- nodo origen de los datos (móvil o embebido);
- parámetros de captura usados en la sesión;
- versión de app/firmware empleada;
- ruta de comunicación utilizada (local o sincronizada);
- resultado de operación (éxito, error o comportamiento degradado);
- número de nodos concurrentes, tasa de envío y tamaño de carga útil en las pruebas de capacidad.

Esta documentación permite reproducibilidad y facilita análisis de incidencias.

6.4 Métricas de evaluación

Las métricas se centran en comportamiento del sistema:

- tasa de sesiones completadas sin error;
- latencia percibida en operaciones de control (start/stop/descarga);
- consistencia de metadatos y formato de sesión;
- estabilidad de conexión en modo embebido;
- esfuerzo operativo (pasos necesarios por flujo);
- degradación de latencia o pérdida de operaciones al aumentar el número de nodos concurrentes.

6.5 Plantilla de comparativa móvil vs embebido

Aspecto	Nodo móvil (Android)	Nodo embebido (ESP32+IMU)	Criterio de evaluación
Flujo de captura	Local y directo	Remoto con control desde app	Compleitud de flujo
Persistencia	Sesión local en app	Descarga y guardado en app	Trazabilidad de sesión
Operación	Interacción simple	Mayor dependencia de integración	Robustez operativa
Estado/diagnóstico	Inmediato en interfaz	Requiere chequeo de conectividad	Tiempo de resolución
Evolución cloud	Integración prevista con backend	Integración prevista vía contratos comunes	Escalabilidad de arquitectura

Cuadro 6.1: Plantilla de comparación entre modo móvil y modo embebido.

6.6 Criterios de aceptación

Se considerará válida la arquitectura en el alcance del TFG cuando:

- el flujo captura-guardado-consulta funcione de forma repetible;
- la integración Android-ESP32 permita control y descarga de sesiones;
- el formato de sesión sea consistente entre orígenes;
- se documenten límites operativos y decisiones de evolución pendientes.

6.7 Resultados

Este apartado se reserva para presentar resultados reales de campaña, distinguiendo claramente entre validación funcional, integración y operación.

6.7.1 Resultados del modo móvil

Incluir sesiones representativas de captura local, tiempos de operación y comportamiento de gestión de sesiones.

6.7.2 Resultados del modo embebido

Documentar resultados de conexión, control de captura y descarga de sesiones desde ESP32, incluyendo incidencias observadas.

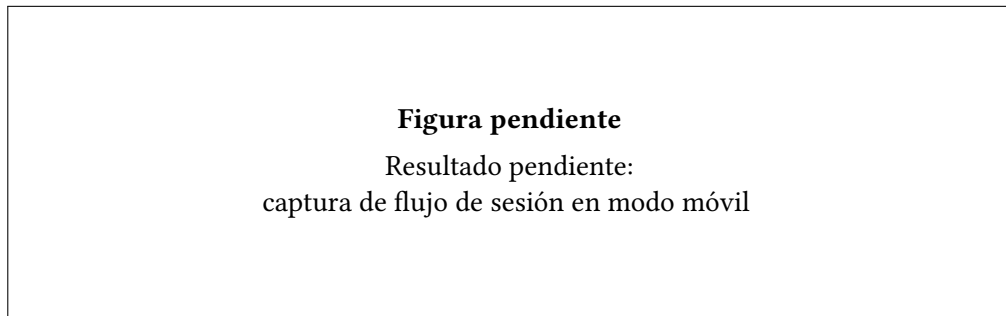


Figura 6.2: Resultados representativos del modo móvil.

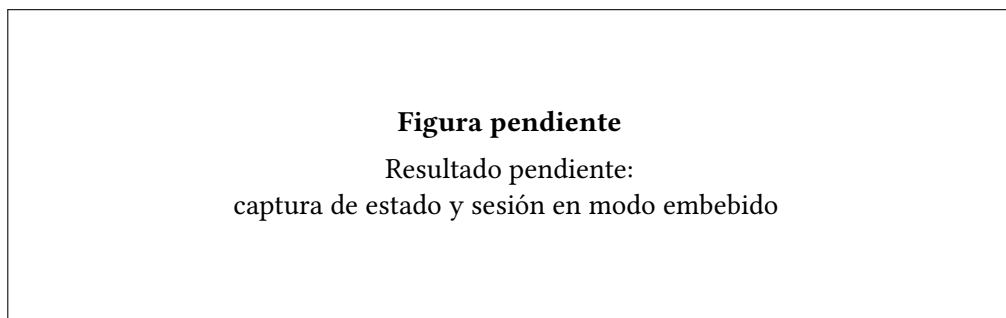


Figura 6.3: Resultados representativos del modo embebido.

6.7.3 Evaluación del subsistema de comunicación

Este bloque debe recoger la comparación entre los mecanismos de comunicación considerados para el enlace Android-ESP32. La evaluación debe centrarse en latencia, estabilidad de operación, simplicidad de integración y comportamiento cuando aumenta la carga de mensajes o el número de nodos concurrentes.

Como guía de redacción, conviene documentar:

- tecnologías comparadas;
- tamaño de mensaje y frecuencia de envío usados en la prueba;
- tiempos de respuesta observados en operaciones de control;
- pérdidas, errores o degradación bajo carga;
- criterio seguido para seleccionar la opción más adecuada.

6.7.4 Resultados de integración por contratos

Este bloque debe mostrar si los contratos de sesión y control se cumplen de forma homogénea y dónde aparecen incompatibilidades.

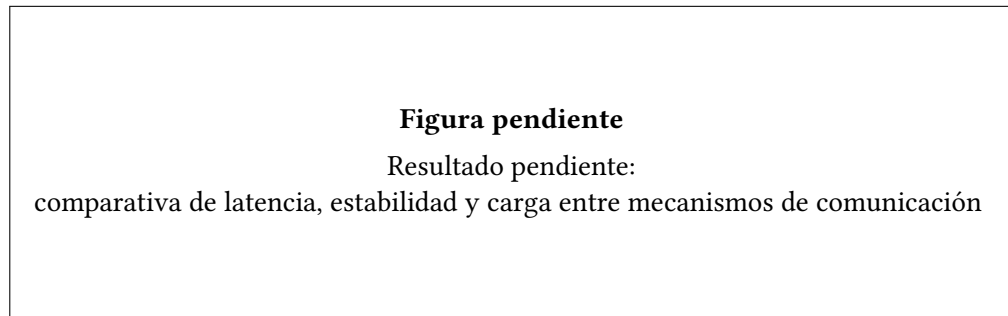


Figura 6.4: Evaluación comparativa del subsistema de comunicación Android-ESP32.

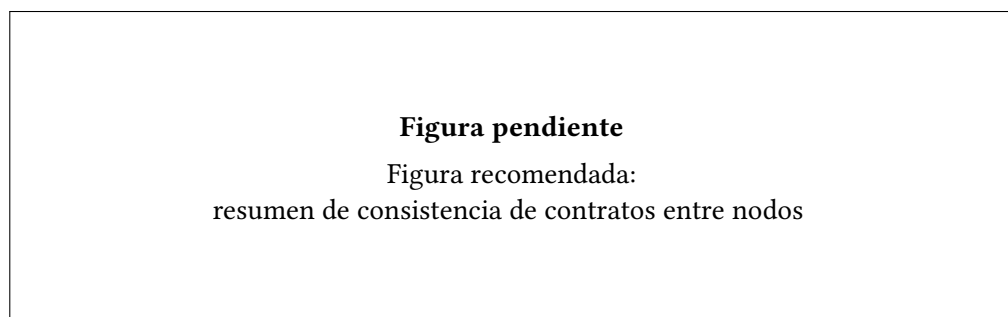


Figura 6.5: Validación de contratos de integración entre subsistemas.

6.7.5 Comparación de operación entre modos

La comparación debe centrarse en esfuerzo operativo, estabilidad de flujo y mantenibilidad, no solo en valores puntuales.

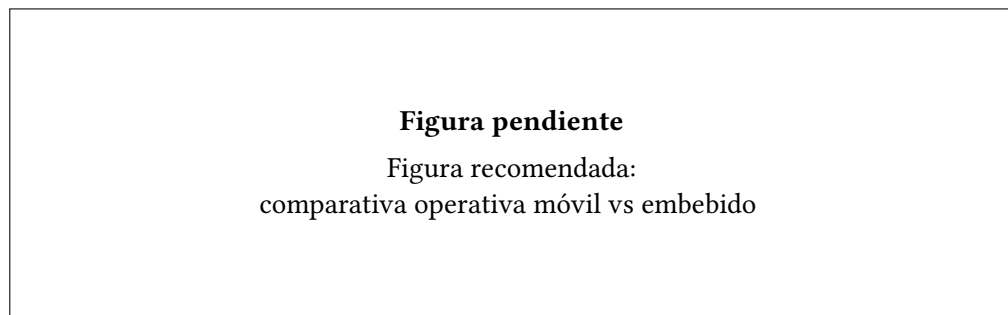


Figura 6.6: Comparación operativa entre modo móvil y modo embebido.

6.8 Discusión

La discusión final debe abordar:

- fortalezas reales del diseño modular;

- puntos de fricción en integración embebida;
- impacto de no tener todavía backend cloud en producción;
- criterio técnico para decidir la evolución de mecanismos de comunicación y backend.

6.8.1 Limitaciones del trabajo

Las limitaciones deben expresarse en términos de alcance de implementación, cobertura de pruebas y grado de madurez de integración.

6.8.2 Lecciones aprendidas

Como cierre de validación, este apartado debe recoger decisiones que hayan reducido riesgo técnico y pautas que mejoren la siguiente iteración del sistema.

Conclusiones

EL trabajo se consolida en torno a una idea principal: construir un sistema de captura inercial capaz de operar con más de una plataforma de adquisición dentro de un mismo flujo de trabajo.

7.1 Conclusiones principales

- Android resulta una base viable para captura, gestión de sesiones y operación de campo en fases tempranas del sistema.
- La separación funcional entre adquisición, comunicación, persistencia y visualización facilita mantenimiento e integración progresiva.
- La incorporación de un nodo ESP32 aporta una ruta realista para adquisición dedicada y para validar la interoperabilidad entre plataformas.
- El estado actual permite un flujo funcional local extremo a extremo y deja preparada la evolución hacia operación remota.

7.2 Conclusión metodológica

La principal conclusión metodológica del TFG es que, en un proyecto de este alcance, resulta más sólido cerrar primero una arquitectura operativa por capas que comprometer desde el inicio una infraestructura definitiva.

Este criterio ha permitido:

- reducir riesgo técnico por integración progresiva;
- mantener trazabilidad entre decisiones y resultados;
- diferenciar claramente funcionalidades implementadas frente a líneas de evolución.

7.3 Implicaciones prácticas

Desde el punto de vista de ingeniería, la solución propuesta permite:

- reutilizar la misma lógica de operación con distintos nodos de adquisición;
- introducir backend remoto sin rediseñar captura ni interfaz;
- evaluar tecnologías de comunicación por evidencia, no por compromiso previo.

7.4 Aportaciones

Las aportaciones del trabajo en su estado actual son:

- definición de una base ciberfísica para captura inercial sobre plataformas heterogéneas;
- implementación de modo móvil operativo con gestión de sesiones;
- integración funcional Android-ESP32 para control y descarga de capturas;
- formalización de contratos de sesión, control y explotación;
- marco de validación orientado a interoperabilidad y operación.

7.5 Trabajo futuro

Las líneas de continuidad más relevantes son:

- integrar una primera capa backend para sincronización de sesiones y estado;
- ejecutar campaña de validación comparativa completa con protocolo homogéneo;
- mejorar trazabilidad automática (versionado de configuración y registro de incidencias);
- evaluar alternativas de comunicación externa si aportan mejoras medibles en operación;
- reforzar automatización de pruebas de integración entre app y nodo embebido.

7.5.1 Línea prioritaria inmediata

La prioridad inmediata es consolidar una primera versión funcional de capa backend y cerrar la validación de integración extremo a extremo sobre esa base.

Apéndices

Apéndice A

Material adicional

EN este apéndice se recoge material de apoyo para reproducir el entorno experimental y el flujo de trabajo técnico del proyecto.

A.1 Scripts principales

- `capture_http_imu.py`: recepción HTTP y guardado de sesiones.
- `process_imu_session.py`: procesado offline con métricas y visualización.
- `process_imu_step_by_step.py`: visualización por etapas del pipeline.

A.2 Comandos de referencia

A.2.1 Captura

```
1 python3 capture_http_imu.py --host 0.0.0.0 --port 8001 --seconds 30
```

A.2.2 Procesado rápido sin gráfica

```
1 python3 process_imu_session.py session_YYYYMMDD_HHMMSS_30s.csv  
   --no-plot
```

A.2.3 Procesado paso a paso con validación temporal

```
1 python3 process_imu_step_by_step.py session_YYYYMMDD_HHMMSS_30s.csv  
   \  
2 --mode step4 --g-fc 0.03 --fmin 0.02 --fmax 1.20 \  
3 --height-min-change 0.00005 --height-step-min 0.000001  
   --confirm-samples 6
```

A.3 Datos de prueba

Se trabajó con múltiples sesiones de 30 segundos en formato CSV, incluyendo capturas de reposo y capturas con movimiento vertical manual. La bitácora de resultados y ajustes se mantiene en `INFORME_IMU.md`.

Bibliografía
